```
n ← length(A)
total ← 0
for i in {0..n-1} do
  total ← total + A[i]
```

After the `for` keyword we will specify the name of the variable (`i`) that will hold the values on which we iterate, then we need the `in` keyword and after that the (ordered) list of values that `i` will assume (in this case, all integers from `0` to `n-1` included).

Since we only need the index `i` to access the current element in the array, we can obtain the same result in a more succinct way by iterating over `A`'s elements directly:

```
total ← 0
for a in A do
  total ← total + a
```

Finally, as we have shown in a previous section, whenever possible we will also use a mathematical notation instead of a `for` loop to be even more succinct, without hurting clarity:

```
total ← sum{A}
```

The last two options are, obviously, not always viable. For instance, if we need to write complex expressions using more than one element of an array at a time, then we might need to explicitly iterate over indices: